

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники

ОТЧЁТ
по задачам: М-Р
по дисциплине «Алгоритмы и структуры данных»

Выполнил:
Ануфриев Андрей Сергеевич
Р3219

Содержание

1	Задание М. Цивилизация	2
2	Задание N. Свинки-копилки	4
3	Задание О. Долой списывание!	6
4	Задание Р. Авиаперелёты	8

1 Задание М. Цивилизация

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
int main() {
    int n, m, sx, sy, fx, fy;
    cin >> n >> m >> sx >> sy >> fx >> fy;
    sx--;
    sy--;
    fx--;
    fy--;
    vector<string> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];
    vector<vector<int>> d(n, vector<int>(m, 1e9));
    vector<vector<pair<int, int>>> p(n, vector<pair<int, int>>(m, {-1, -1}));
    priority_queue<pair<int, pair<int, int>>> q;
    d[sx][sy] = 0;
    q.push({
        0, {sx, sy}
    });
    int dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, 1, -1};
    char dir[] = {'N', 'S', 'E', 'W'};
    while (!q.empty()) {
        int x = q.top().second.first;
        int y = q.top().second.second;
        q.pop();
        for (int i = 0; i < 4; i++) {
            int nx = x + dx[i], ny = y + dy[i];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m)
                continue;
            if (a[nx][ny] == '#')
                continue;
            int cost = (a[nx][ny] == '.') ? 1 : 2;
            if (d[x][y] + cost < d[nx][ny]) {
                d[nx][ny] = d[x][y] + cost;
                p[nx][ny] = {x, y};
                q.push({
                    -d[nx][ny], {nx, ny}
                });
            }
        }
    }
    if (d[fx][fy] == 1e9) {
        cout << -1;
        return 0;
    }
    string path;
    int x = fx, y = fy;
    while (x != sx || y != sy) {
        int px = p[x][y].first, py = p[x][y].second;
        for (int i = 0; i < 4; i++) {
```

```

        if (px + dx[i] == x && py + dy[i] == y) {
            path += dir[i];
            break;
        }
    }
    x = px;
    y = py;
}
reverse(path.begin(), path.end());
cout << d[fx][fy] << "\n" << path;
}

```

Суть решения

Основная проблема: найти путь минимальной стоимости между двумя точками на карте с разнородными рельефами, где каждый тип рельефа имеет свою стоимость прохода.

Ключевая идея: это классическая задача поиска кратчайшего пути с взвешенными рёбрами. Поскольку веса рёбер положительны (1 для поля, 2 для леса), применяем алгоритм Дейкстры. Идея состоит в том, чтобы всегда расширять путь из узла с минимальной известной стоимостью, гарантируя, что это оптимальный путь до этого узла.

1. **Выбор оптимального направления:** приоритетная очередь обеспечивает выбор вершины с минимальным расстоянием, избегая неоптимальных путей.
2. **Быстрая релаксация:** благодаря max-heap (использованием отрицательных значений) достигаем эффективной работы с $O(\log nm)$ на операцию.
3. **Восстановление пути:** сохраняем родительскую вершину для каждой достигнутой позиции, что позволяет восстановить весь маршрут в конце.
4. **Обработка непроходимых препятствий:** вода блокируется на уровне проверки соседей, неподходящие маршруты просто не рассматриваются.

Время: $O(nm \log nm)$

Память: $O(nm)$

2 Задание N. Свинки-копилки

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> keyIn(n);
    for (int i = 0; i < n; i++) {
        cin >> keyIn[i];
        keyIn[i]--;
    }
    vector<int> state(n, 0);
    int cycles = 0;
    for (int i = 0; i < n; i++) {
        if (state[i] == 0) {
            int cur = i;
            while (state[cur] == 0) {
                state[cur] = 1;
                cur = keyIn[cur];
            }
            if (state[cur] == 1) {
                cycles++;
            }
            cur = i;
            while (state[cur] != 2) {
                state[cur] = 2;
                cur = keyIn[cur];
            }
        }
    }
    cout << cycles << endl;
    return 0;
}
```

Суть решения

Основная проблема: найти минимальное количество копилочек для разбиения, чтобы получить доступ ко всем деньгам. Это эквивалентно подсчёту циклов в функциональном графе, так как каждый цикл требует разбиения ровно одной копилки.

Ключевая идея: рассматриваем ключи как функциональный граф — для каждой копилки i есть ровно одно исходящее ребро в копилку $keyIn[i]$. Граф из n вершин и n рёбер распадается на несвязные компоненты, каждая из которых содержит ровно один цикл (потому что каждая вершина имеет одно входящее и одно исходящее ребро). Нужно подсчитать количество этих циклов.

1. **Обнаружение цикла:** использование двух состояний (в текущем пути = 1, полностью обработана = 2) позволяет точно определить, когда мы замкнулись в цикл, в отличие от входа в уже обработанную компоненту.
2. **Избежание переподсчёта:** каждая компонента рассматривается ровно один раз благодаря маркировке вершин на этап 2 после прохода.

3. **Линейная сложность:** каждая вершина посещается максимум дважды, обеспечивая оптимальное время работы даже на больших входах.

Время: $O(n)$

Память: $O(n)$

3 Задание О. Долой списывание!

```
#include <iostream>
#include <vector>
using namespace std;
vector<vector<int>>> g;
vector<int> color;
bool ok = true;
void dfs(int v, int c) {
    color[v] = c;
    for (int to : g[v]) {
        if (color[to] == 0) {
            dfs(to, 3 - c);
        } else if (color[to] == c) {
            ok = false;
            return;
        }
    }
}
int main() {
    int n, m;
    cin >> n >> m;
    g.resize(n + 1);
    color.assign(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }
    for (int i = 1; i <= n; i++) {
        if (color[i] == 0) {
            dfs(i, 1);
        }
    }
    cout << (ok ? "YES" : "NO");
    return 0;
}
```

Суть решения

Основная проблема: разделить студентов на две группы (списывающие и дающие) так, чтобы каждый обмен записками был между студентами из разных групп.

Ключевая идея: это задача проверки двудольности графа. Граф двудольный тогда и только тогда, когда его можно покрасить в два цвета так, что соседние вершины имеют разные цвета. Это эквивалентно отсутствию циклов нечётной длины. Используем DFS с двухцветной раскраской — если разумею когда-либо присвоить соседу цвет, отличный от требуемого, граф не двудольный.

1. **Правильная раскраска соседей:** формула $3 - c$ — с гарантирует чередование ровно двух цветов при спуске по DFS.
2. **Обнаружение противоречия:** проверка существующего цвета вершины позволяет тут же обнаружить нарушение двудольности без необходимости переопроверки.

3. **Связные компоненты:** обработка всех непосещённых вершин гарантирует проверку всех компонент связности графа.

Время: $O(n + m)$

Память: $O(n + m)$

4 Задание Р. Авиаперелёты

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;
int n;
vector<vector<int>> w;
bool stronglyConnected(int C) {
    vector<vector<int>> adj(n), radj(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i != j && w[i][j] <= C) {
                adj[i].push_back(j);
                radj[j].push_back(i);
            }
        }
    }
    vector<bool> visited(n, false);
    auto dfs = [&](int v, const vector<vector<int>>& graph, auto&& self) ->
        void {
            visited[v] = true;
            for (int to : graph[v]) {
                if (!visited[to]) {
                    self(to, graph, self);
                }
            }
        };
    dfs(0, adj, dfs);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            return false;
    }
    fill(visited.begin(), visited.end(), false);
    dfs(0, radj, dfs);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            return false;
    }
    return true;
}

int main() {
    cin >> n;
    w.assign(n, vector<int>(n));
    int maxWeight = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> w[i][j];
            maxWeight = max(maxWeight, w[i][j]);
        }
    }
    int left = 0, right = maxWeight;
    while (left < right) {
        int mid = (left + right) / 2;
```

```

        if (stronglyConnected(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    cout << left << endl;
    return 0;
}

```

Суть решения

Основная проблема: найти минимальный размер топливного бака, достаточный для перелёта из любого города в любой другой (с дозаправками).

Ключевая идея: если зафиксировать размер бака C , можно построить граф всех возможных перелётов (рёбра включаются, если расход топлива $\leq C$). Нужна минимальная величина C , при которой граф остаётся сильносвязным. Ответ монотонен: если граф сильносвязен при C , то сильносвязен и при $C' > C$. Применяем бинарный поиск по C с проверкой сильносвязности.

1. **Проверка сильносвязности:** используем два DFS на прямом и обратном графе. Если из города 0 достижимы все города в прямом графе, и все города могут достичь город 0 в обратном графе, то граф сильносвязен.
2. **Монотонность:** поиск оптимального значения гарантирует нахождение минимума благодаря монотонности функции сильносвязности.
3. **Эффективность:** бинарный поиск снижает количество проверок с maxWeight до $\log(\text{maxWeight})$.

Время: $O(n^2 \log(\text{maxWeight}))$

Память: $O(n^2)$